

FRC Java Coding Lab 7.0 - Final Frozen Package Backbone

Permanent architecture contract for Inheritance Development

1. Backbone Contract

Driver -> Xbox Controller -> controls -> commands -> subsystems -> io -> Hardware

telemetry observes any stage without controlling the pipeline

FINAL FROZEN STATUS. Future lessons may extend this architecture, but may not redesign its responsibilities, dependency direction, control flow, or observation flow without a formal architecture review.

2. Package Responsibilities

Package	Primary question	Responsibility	Examples	Must not contain
controls	What does the driver want?	Convert raw human input into processed robot intent.	Deadband, inversion, scaling, curves, slew-rate limiting.	Motor controllers, CAN IDs, CommandScheduler, hardware logic.
commands	What should the robot do now?	Coordinate one robot action or action sequence.	DefaultDriveCommand, ShootCommand, AutoBalanceCommand.	Vendor configuration, motor creation, low-level hardware handling.
subsystems	What can this mechanism do?	Provide a high-level mechanism API and own current mechanism state.	tankDrive(), stop(), runIntake(), setShooterSpeed().	Button mappings, Xbox access, direct vendor configuration.
io	How does the hardware work?	Define hardware contracts; implementations contain vendor APIs and populate observation snapshots.	DriveIO, DriveIOInputs, DriveIOSparkMax, DriveIOSim, GyroIO.	Command coordination, driver-intent processing, NetworkTables publishers.
telemetry	What is the robot doing?	Publish live observations and diagnostics without changing behavior.	RobotTelemetry, DriveTelemetryFacade, typed NT4 publishers.	Motor control, input processing, scheduling, vendor hardware APIs.
util	Is it truly shared?	Contain generic helpers reused across multiple mechanisms.	Math, units, generic diagnostics and fault helpers.	Drive-, intake-, shooter-, or mechanism-specific logic.

3. Stable Project Tree

```
src/main/java/frc/robot
|- Main.java
|- Robot.java
|- RobotContainer.java
|- Constants.java
|- commands/
|  |- drive/
|  |- intake/
|  |- shooter/
|  `~ auto/
|- controls/
|- subsystems/
|- io/
|  |- drive/
|  |  |- DriveIO.java
|  |  |- DriveIOSparkMax.java
|  |  `~ DriveIOSim.java
|  |- gyro/
|  `~ vision/
|- telemetry/
|  |- RobotTelemetry.java
|  `~ drive/
```

```

| | - DriveTelemetryFacade.java
| | - DriveInputTelemetry.java
| | - DriveHardwareTelemetry.java
| | - DriveSensorTelemetry.java
| | ~ DrivePoseTelemetry.java
| ~ util/

src/main/deploy/pathplanner/
| - paths/
| ~ autos/

```

4. Non-Negotiable Dependency Rules

- RobotContainer is a concise composition root: create objects, select implementations, inject dependencies, configure commands and bindings.
- RobotContainer contains no input-processing logic, no periodic telemetry publishing, and no hardware behavior.
- Commands depend on subsystems and the smallest telemetry facade they actually use.
- Subsystems depend on IO interfaces, never on vendor implementations.
- IO implementations may use vendor APIs; IO interfaces and Inputs snapshots must not depend on NetworkTables.
- Each IO interface owns a mechanism-specific Inputs snapshot. DriveIOInputs is the drive implementation of this pattern.
- Hardware observations originate in IO. Mechanism state originates in a subsystem or dedicated estimator.
- Telemetry publishes snapshots; it never reads vendor devices directly and never commands robot behavior.
- DriveTelemetryFacade hides internal telemetry modules; callers do not navigate deep getter chains.
- util remains generic. A helper used by only one mechanism stays with that mechanism.

5. Approved Flows

CONTROL

Driver -> Controller -> controls -> command -> subsystem -> IO -> hardware

OBSERVATION

Controller / input processor -> DriveInputTelemetry

IO.updateInputs(MechanismIOInputs) -> subsystem periodic -> mechanism telemetry facade

Subsystem / estimator state -> pose or state telemetry

FORBIDDEN

Telemetry -X-> control / scheduling / vendor API

IO -X-> NetworkTables / command coordination

RobotContainer -X-> periodic business logic

6. Drive Telemetry Contract

- DriveInputTelemetry: raw, processed, and commanded driver-pipeline values.
- DriveHardwareTelemetry: applied output, voltage, current, temperature, connection, and fault observations from DriveIOInputs.
- DriveSensorTelemetry: encoder and gyro observations supplied through approved IO/state contracts.
- DrivePoseTelemetry: pose and odometry state; it never computes odometry.
- DriveTelemetryFacade: the only public access point for drive telemetry.

7. Constants Policy

- Constants.java remains the default configuration authority.
- Organize constants by mechanism or concern using nested static classes such as DriveConstants, OperatorConstants, AutoConstants, SensorConstants, and TelemetryConstants.
- Do not split Constants.java during normal lessons.
- A constants/ package may be introduced only through a formal architecture review when project complexity clearly justifies it.

8. Autonomous and PathPlanner Policy

- commands/auto coordinates autonomous actions, command compositions, factories, and named-command registration.
- PathPlanner resources belong in src/main/deploy/pathplanner/paths and src/main/deploy/pathplanner/autos.
- Autonomous commands do not own pose, odometry, state estimation, or coordinate state.

- Pose and estimation responsibilities belong to the drive subsystem or a dedicated estimator.

9. Backbone Extension Policy

- The backbone is frozen, but extensible.
- New mechanisms must follow the approved responsibility boundaries, dependency direction, control flow, and observation flow.
- Create only classes that have real responsibilities.
- Follow the drive responsibility pattern; do not mechanically duplicate every drive class.

10. Per-Mechanism IO Inputs Pattern

- Every IO interface owns its own mechanism-specific Inputs snapshot.
- Examples: DriveIOInputs, IntakeIOInputs, ShooterIOInputs, ElevatorIOInputs, VisionIOInputs, and GyroIOInputs.
- IO implementations populate observations; subsystems consume the snapshots; telemetry publishes them.
- Telemetry never reads a motor controller directly, and subsystems never bypass IO to read a CAN device.

11. Final Change Policy

Allowed: add a class inside the correct package, extend a mechanism Inputs snapshot, add a facade method, or add a new mechanism using the same approved pattern.

Not allowed in normal lessons: move responsibilities between packages, reverse dependency direction, let telemetry control behavior, let IO publish NetworkTables, or grow RobotContainer into a logic class.

FINAL LOCKED - NO FURTHER BACKBONE CHANGES